A Morse Code-based Optical Communication System

Kartik Lahoti, Puni Aditya and G V V Sharma
Department of Electrical Engineering
IIT Hyderabad

***Abstract*—This paper outlines a comprehensive multi-stage development of a Morse Code On-Off Keying (OOK) system. We first establish a baseline Visible Light Communication (VLC) link to demonstrate the fundamental principles of communication over an optical link. Subsequently, we introduce Infrared (IR) communication to extend the range and explore more advanced techniques to improve the system performance. This is done by using the Arduino as the transceiver platform. Our communication scheme is successfully demonstrated for a range of up to 2 meters.**

***Index Terms*—Visible Light Communication (VLC), Infrared (IR), Deynamic Thresholding, Goertzel Algorithm, On-Off Keying (OOK), Adaptive Thresholding.**

I. Introduction

Wireless optical communication is highly dependent on clear Line-of-Sight (LoS) and a high Signal-to-Noise Ratio (SNR) [1], [2]. In low-cost embedded systems, a standard Light Emitting Diode (LED) paired with a Light Dependent Resistor (LDR) is commonly used to demonstrate data transfer. However, ambient environmental lighting introduces a severe DC bias into the receiver, and LDRs exhibit significant photoconductive latency [3]. The novelty of this paper lies in its evolutionary approach to solving physical layer limitations, building upon foundational On-Off Keying (OOK) optical paradigms [4]. We first establish a baseline VLC system to document hardware-induced signal degradation, known as the "Iceberg Effect." Following this, we introduce an advanced Infrared (IR) iteration utilizing Dynamic Thresholding—a software-defined algorithm that tracks the environmental noise floor [5]. Finally, to overcome severe outdoor solar interference [6], we introduce a Goertzel algorithm-based digital signal processing (DSP) approach, effectively turning a low-cost microcontroller into a software-defined radio for optical frequencies.

A. Related Works and Novelty

Standard commercial IR systems avoid ambient noise by utilizing 38kHz Carrier Waves and specialized TSOP receiver modules. While effective, these modules act as "black boxes" that strip the analog signal data away from the microcontroller, preventing custom low-level signal analysis. Conversely, standard unmodulated photodiode circuits use fixed analog thresholds that fail the moment room lighting changes. Our proposed Dynamic Thresholding and Goertzel implementations bridge this gap, retaining raw analog sampling while applying dynamic digital filters to reject ambient DC bias. The related block diagram is available in Fig. 1

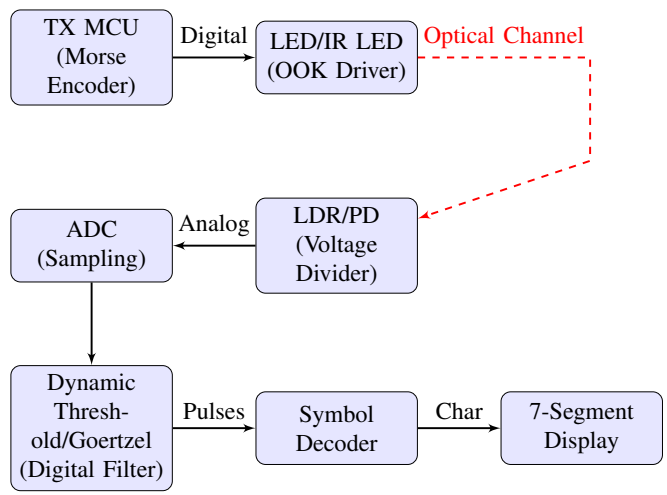

Fig. 1. System Block Diagram detailing the cross-layer flow from hardware transmission to the software-defined filtering and hardware output.

To ensure reproducibility, the system was tested using standard off-the-shelf components. The development was split into two hardware profiles as detailed in Table I.

TABLE I
List of Components

Setup	Components
Common	2× Arduino UNO, 2× Breadboards, Resistors, Jumper Cables
Visible	1× Red LED (5mm), 1× LDR (5mm)
IR	1× 940-nm IR LED, 1× IR Photodiode (Reverse Bias)

B. Baud Rate and Telemetry Applicability

Morse code, utilizing OOK, possesses a fundamentally low baud rate compared to modern digital modulation schemes like UART or QAM. However, for specialized telemetry applications—such as underwater optical communication or low-cost sensor nodes—signal penetration and robustness take

precedence over raw speed. The widened tolerance windows used in this paper's decoding logic demonstrate how low baud rate systems can reliably reconstruct highly degraded signals [7].

II. System Model

A. Baseband Morse Encoding

To establish a quantitative baseline for On-Off Keying (OOK) modulation, the system utilizes the standard international Morse code protocol. In this telemetry model, the transmitter's digital output pin toggles between a HIGH (active) and LOW (idle) state to construct characters.

The protocol defines all signal durations relative to a single base time unit (T_u), which in our experimental setup is set to 200 ms. The strict timing rules governing this variable-length baseband modulation are detailed in Table II.

TABLE II
Standard Morse Protocol Timings ($T_u = 200$ ms)

Protocol Element	Logic State	Duration
Dot (Dit)	HIGH (ON)	1 T_u (200 ms)
Dash (Dah)	HIGH (ON)	3 T_u (600 ms)
Intra-character gap	LOW (OFF)	1 T_u (200 ms)
Inter-character gap	LOW (OFF)	3 T_u (600 ms)
Inter-word gap	LOW (OFF)	7 T_u (1400 ms)

This baseband square wave serves as the fundamental logic envelope for all subsequent experimental phases. A visual representation of these timing constraints is illustrated in Fig. 2 for the transmission of the word "AN".

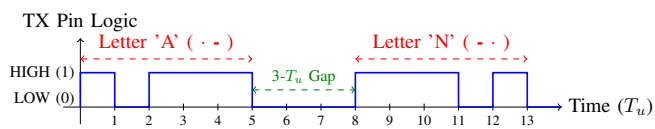

Fig. 2. Baseband OOK transmission waveform for the word "AN", illustrating the strict 1:3 timing ratio of standard Morse protocol.

B. Hardware Setup

Fig. 3 shows the transceiver configuration used for various cases.

- Transmitter (TX): An Arduino microcontroller drives a standard visible/IR LED. The LED's anode connects to Digital Pin 5, while the cathode is grounded through a 220Ω current-limiting resistor.
- Receiver (RX): The optical sensor is a Light Dependent Resistor (LDR)/IR receiver (connected in reverse bias) arranged in a voltage divider. The 5V supply is connected to one leg of the receiver. The secondary leg connects to the Arduino's Analog Pin A0 and a pull-down resistor (R) connected to ground.

The corresponding details are available in Table III.

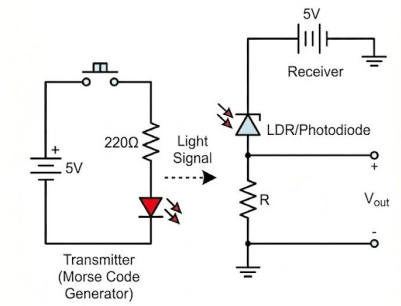

Fig. 3. Circuit Diagram

TABLE III
Maximum Reliable Distance vs. Load Resistance

Configuration	Load Resistor (R)	Maximum Range
Visible Light (LDR)	10 kΩ	27 cm
Baseband IR	10 kΩ	≈ 50 cm (47 cm)
Baseband IR	1 MΩ	≈ 2 m (157 cm)
Frequency-Modulated IR	1 MΩ	> 2 m (Slightly better)

III. Communication System

A. Visible Light Baseline

When bridging analog optical sensing and digital microcontrollers, signal truncation occurs. As shown in Fig. 4, an expected "Dah" signal transmitted for 600ms is detected for a fraction of that time. LDRs exhibit notable rise/fall latency, resulting in a trapezoidal curve. To counteract this, our receiver employs widened tolerance windows as listed in Table IV. The baseline system achieved a maximum reliable transmission distance of only 11-12 cm in room light, and 27 cm in total darkness.

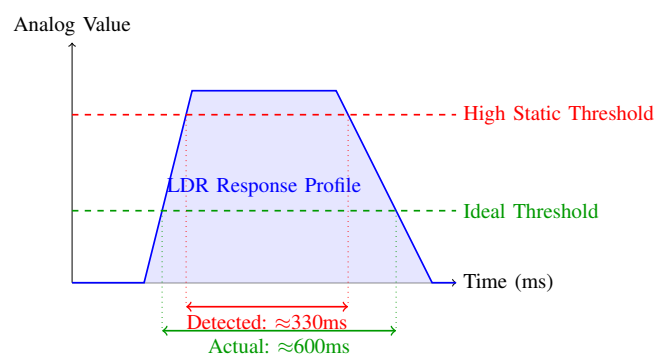

Fig. 4. The "Iceberg Effect" illustrating temporal truncation caused by improper static thresholding of an LDR signal.

TABLE IV
ADAPTIVE MORSE PROTOCOL TIMINGS TO COMBAT "ICEBERG EFFECT"

Protocol Element	Logic State	Duration
Dot (Dit)	HIGH (ON)	150 ms
Dash (Dah)	HIGH (ON)	450 ms
Intra-character gap	LOW (OFF)	350 ms
Inter-character gap	LOW (OFF)	750 ms
Inter-word gap	LOW (OFF)	1600 ms

B. Infrared Evolution and Dynamic Thresholding

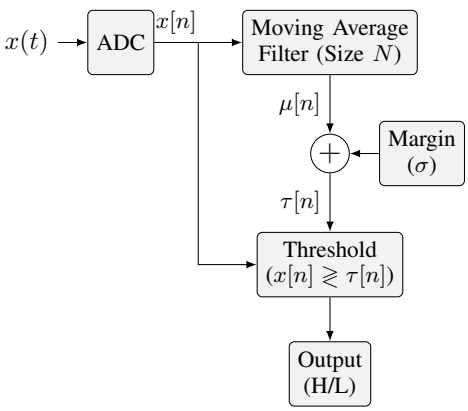

Fig. 5. Block Diagram of Receiver used in IR with Dynamic Thresholding

1) Hardware Optimization and Adaptive Filtering: Operating in reverse-bias, the photodiode produces a photocurrent (I_{pd}) proportional to the incident photon flux; consequently, the voltage sampled by the ADC is governed by $V_{adc} = I_{pd} \times R$. By increasing R from 10 kΩ to 1 MΩ, sensitivity was maximized, extending the raw range to 1.57 meters indoors.

To isolate transmission pulses from fluctuating indoor IR noise, we implement a discrete-time Dynamic Thresholding algorithm. The core challenge is that the photodiode's ambient baseline is not zero; it is a floating DC offset that shifts as environmental lighting changes. To track this, the receiver converts the continuous-time analog voltage $x(t)$ into a discrete-time sequence by polling the Analog-to-Digital Converter (ADC) at designated intervals. Let T_s represent this sampling period. The discrete sample index n maps directly to physical continuous time via the relation $t = nT_s$, defining our sampled sequence as $x[n] = x(nT_s)$.

The algorithm calculates a rolling local mean, over a circular buffer of N samples to capture the slow-moving ambient noise floor:

$$\tau[n] = \frac{1}{N} \sum_{i=0}^{N-1} x[n-i] + \sigma \tag{1}$$

This moving average inherently acts as a digital low-pass filter. To extract the high-frequency OOK pulses, a dynamic threshold $\tau[n]$ is established by adding a constant margin σ to the local mean:

By dynamically adjusting the threshold relative to the moving baseline, the detection zone floats alongside ambient light shifts. The logic implementation is detailed in Algorithm 1, and the discrete sampling behavior is visualized in Fig. 6.

Algorithm 1 Dynamic Thresholding for OOK Demodulation

Require: Discrete input sequence $x[n]$, Window size N, Margin σ
Initialize circular buffer B of size N to 0
Initialize running sum $S \leftarrow 0$
for each strictly timed sampling interval n **do**
  $S \leftarrow S - B[n \bmod N]$ ▷ Remove oldest sample
  $B[n \bmod N] \leftarrow x[n]$ ▷ Insert new sample
  $S \leftarrow S + x[n]$
  $\mu[n] \leftarrow S/N$ ▷ Calculate local mean
  $\tau[n] \leftarrow \mu[n] + \sigma$ ▷ Set floating threshold
  if $x[n] > \tau[n]$ **then**
    Register logical HIGH (Pulse active)
  else
    Register logical LOW (Idle)
  end if
end for

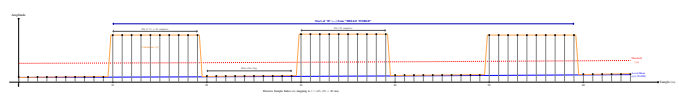

Fig. 6. Exact mathematical mapping of the "HELLO WORLD" sequence. With a base time unit $T_u = 200$ ms and sample interval $T_s = 20$ ms, each standard Dit and Gap captures precisely 10 discrete samples. Because the buffer size is massive ($N = 500$), the local mean $\mu[n]$ remains highly stable and does not climb high enough to clip the signal during pulses, completely preventing baseline wander.

2) Dynamic Thresholding Initialization and Training Sequence: Because the rolling buffer requires a stabilization period to accurately map the ambient noise floor, the system mandates an initialization phase prior to data transmission. Initially, the system starts with an assumed, pre-defined threshold for the signal margin (σ), detailed in Table V, to bootstrap the detection logic based on the expected ambient environment. During this initialization phase, a known training sequence—the phrase "HELLO WORLD"—is transmitted 10 times.

TABLE V
INITIAL ASSUMED SIGNAL MARGINS (σ) PRIOR TO STABILIZATION (1 MΩ SETUP)

Ambient Environment	Assumed Margin (σ)	ADC Voltage Eqv. (5V)
Total Darkness	30	≈ 0.15V
Standard Indoor Lighting	80	≈ 0.39V
High Ambient (Near Window)	150	≈ 0.73V

Based on the timing ratios defined in Table II, a single transmission of "HELLO WORLD" consumes exactly 111 T_u (inclusive of all intra-character, inter-character, and inter-word

gaps). Therefore, the full 10-cycle training sequence spans 1110 T_u. Given our baseline $T_u = 200$ ms, this equates to a deliberate 222-second pre-transmission window. This ensures the Dynamic Thresholding buffer (V_{noise}) fully saturates with representative environmental data, allowing the dynamic baseline to settle completely alongside the assumed margin before the receiver attempts to decode unknown payload characters.

IV. Amplitude Modulation

A. Limitations of Baseband Detection in Sunlight

While unmodulated IR transmission functions indoors, it fails in direct sunlight. The sun acts as a continuous, high-intensity IR source that drives a massive steady-state photocurrent, saturating the receiver's 10-bit ADC and eliminating the dynamic range needed to detect Morse pulses.

To prevent this "hard" saturation outdoors, the receiver's load resistor can be decreased (e.g., from 1MΩ to 10kΩ). While this keeps the solar DC component within measurable bounds, it severely degrades the signal-to-noise ratio (SNR), reducing the intentional pulses to minute fluctuations on a massive DC floor. Recovering this attenuated signal requires transitioning from baseband amplitude tracking to frequency-selective filtering.

B. Modulation

To isolate the data from the solar baseline, we implement On-Off Keying (OOK), a fundamental form of Amplitude Modulation (AM). Instead of emitting a static DC pulse, the transmitter modulates the IR LED with a 2 kHz square wave carrier during every active Morse symbol.

This modulation shifts the transmission into the frequency domain:

- **DC Offset (Sun):** Solar interference remains concentrated at 0 Hz.
- **Data Signal:** The Morse pulses are nested inside a 2000 Hz carrier band.

By toggling the LED at 2 kHz, the receiver can selectively filter out the 0 Hz solar bias and evaluate only the oscillating AC energy. Operating at a 50% duty cycle also halves the active transmission power, extending battery life for remote nodes.

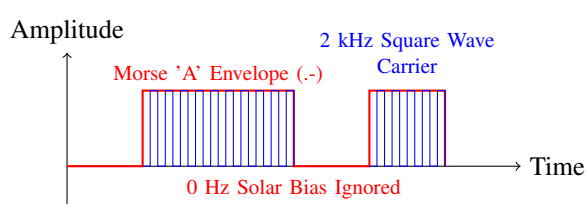

Fig. 7. Amplitude Modulation: The Morse envelope gates a 2 kHz subcarrier, enabling frequency-selective filtering at the receiver.

C. Demodulation

1) Receiver: To detect the 2 kHz carrier digitally, the ADC prescaler is reduced from 128 to 16, accelerating the sampling rate (f_s) to ≈ 60 kHz. This massive oversampling provides the resolution necessary for the Goertzel algorithm [8], which we utilize as a computationally lightweight digital bandpass filter.

The receiver pipeline is illustrated in Figure 8. For a given target frequency index k and block size N, the algorithm computes the intermediate state $y[n]$ for each incoming sample $x[n]$ using the second-order difference equations [9]:

$$s[n] = x[n] + 2\cos\left(\frac{2\pi k}{N}\right) s[n-1] - s[n-2] \quad (2)$$

$$y[n] = s[n] - e^{-j\omega_0} s[n-1] \quad (3)$$

where $y[-1] = y[-2] = 0$.

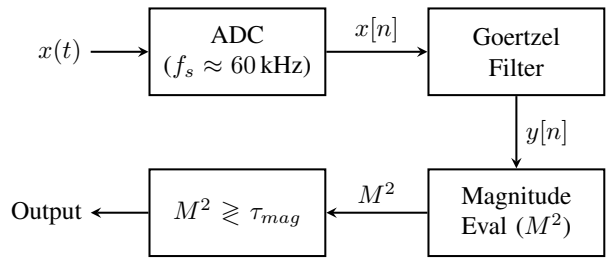

Fig. 8. Block Diagram for Modulated IR Receiver ($\tau_{mag} = 15000$)

After processing N samples, the squared spectral magnitude M^2 at the target frequency is extracted:

$$M^2 = y^2[N-1] + y^2[N-2] \quad (4)$$

$$- 2\cos\left(\frac{2\pi k}{N}\right) y[N-1]y[N-2] \quad (5)$$

Using the squared magnitude bypasses the computational cost of floating-point square roots.

A block size of $N = 80$ provides a processing window of ≈ 1.33 ms, ensuring temporal responsiveness for decoding Morse timings. This creates a bin resolution ($\Delta f = f_s/N$) of 750 Hz. The index $k = \text{round}(N \cdot f_{target}/f_s) = 3$ centers the filter at 2250 Hz. This wide main lobe comfortably captures the 2000 Hz carrier while aggressively suppressing out-of-band signals like 0 Hz DC sunlight and 120 Hz AC flicker.

2) Static Thresholding and OOK Demodulation: While M^2 theoretically approaches zero during transmission gaps, ADC quantization and thermal noise establish a low-level baseline. We apply an empirical static threshold ($\tau_{mag} = 15000$) to digitize the signal.

Because constructive interference from a valid 2 kHz carrier produces M^2 values on the order of 10^5, this threshold safely clears the noise floor while maintaining sensitivity to distant transmissions. The real-time OOK demodulation operates as a binary slicer:

$$\text{Pulse Detected} = \begin{cases} \text{True (HIGH)}, & \text{if } M^2 > \tau_{mag} \\ \text{False (LOW)}, & \text{otherwise} \end{cases} \quad (6)$$

By filtering out the wandering DC bias mathematically, the Goertzel algorithm ensures absolute baseline stability. This eliminates the need for complex adaptive thresholding and allows robust OOK demodulation under variable lighting conditions.

V. Experimental Results

A. Throughput and Bitrate Analysis

We evaluated the efficiency of the three methods by transmitting a 44-byte (352-bit) test sentence. While the VLC setup required slower custom timings (101.8s) to account for photo-conductive latency, the transition to standard Morse timing in IR and Frequency-Modulated IR communication significantly improved throughput. Because Morse code utilizes variable-length OOK encoding, instantaneous throughput fluctuates based on character transmission length. A 44-byte pangram was explicitly chosen as the test payload to force the transmission of all alphabet symbol variants, thereby providing a normalized, worst-case representative average for system bitrate.

TABLE VI
System Performance Comparison (44-Byte Sentence)

Receiver Mode of Communication	**Time (s)**	**Bitrate (bps)**	**Environment**
Visible Light	101.8	3.46	Dark/Indoor only
Baseband IR	81.4	4.32	Indoor (Non-Solar)
Frequency-Modulated IR	81.4	4.32	**Direct Sunlight**

As shown in Table VI, Frequency-Modulated IR maintains the peak bitrate of 4.32 bps even in high-solar environments where VLC and Baseband IR systems experience complete signal saturation and $< 10\%$ decoding accuracy.

B. Range Analysis

The integration of impedance-tuned hardware with software-defined filtering yielded profound improvements. As illustrated in Fig. 9, the visible light baseline failed abruptly at 27 cm. The unoptimized IR setup (10kΩ) reached 47 cm. The fully optimized IR-Dynamic Thresholding setup (1MΩ) successfully maintained 100% decoding accuracy up to 157 cm indoors. Decoding accuracy was empirically measured as the percentage of correctly reconstructed characters from the 44-byte test payload at varying distances.

VI. Conclusion and Future Work

This paper demonstrated a systematic progression in overcoming the physical layer limitations of low-cost optical wireless communication. By evolving from basic visible light sensing to an adaptive Infrared system, we mitigated photo-conductive latency and indoor thermal noise. By shifting the complexity from hardware to a software-defined paradigm—utilizing 2 kHz OOK modulation and an optimized Goertzel digital filter—the system successfully maintained 100% decoding accuracy at ranges up to 2m, even under direct solar interference. This "virtual filter" approach highlights the potential for low-power microcontrollers to perform sophisticated signal recovery in high-noise environments without requiring specialized commercial hardware. Future work will explore real-time transmission and increasing the baud rate, communication range as well as the security.

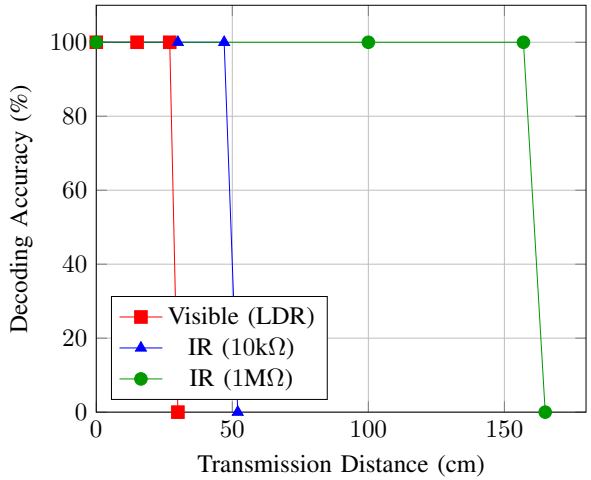

Fig. 9. Comparative analysis of decoding accuracy over distance across the configurations.

References

[1] H. Elgala, R. Mesleh, and H. Haas, "Indoor optical wireless communication: potential and state-of-the-art," *IEEE Communications Magazine*, vol. 49, no. 9, pp. 56–62, Sep 2011.

[2] Z. Ghassemlooy, W. Popoola, and S. Rajbhandari, *Optical Wireless Communications: System and Channel Modelling with MATLAB*. Boca Raton, FL, USA: CRC Press, 2012.

[3] A. Sewaiwar, S. V. Tiwari, and Y. Chung, "Smart led-based ambient light adaptive vlc system," *Optics Express*, vol. 23, no. 19, pp. 24 456–24 465, 2015.

[4] T. Komine and M. Nakagawa, "Fundamental analysis for visible-light communication system using led lights," *IEEE Transactions on Consumer Electronics*, vol. 50, no. 1, pp. 100–107, Feb 2004.

[5] R. J. Green, H. Joshi, M. D. Higgins, and M. S. Leeson, "Adaptive thresholding for optical wireless receivers," *IEE Proceedings - Optoelectronics*, vol. 152, no. 6, pp. 317–321, Dec 2005.

[6] S. Rajagopal, R. D. Roberts, and S.-K. Lim, "Ieee 802.15.7 visible light communication: modulation schemes and dimming support," *IEEE Communications Magazine*, vol. 50, no. 3, pp. 72–82, Mar 2012.

[7] M. Uysal, F. Miramirkhani, T. Baykas, and K. Qaraqe, "Visible light communication for vehicular networking: Performance study of a v2v system using a low-complexity receiver," *IEEE Vehicular Technology Magazine*, vol. 10, no. 4, pp. 71–79, Dec 2015.

[8] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA, USA: California Technical Publishing, 1997.

[9] Wikipedia contributors. (2024) Goertzel algorithm. [Online]. Accessed: May 20, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Goertzel_algorithm

Appendix

1) The code can be uploaded to the Arduino UNO using `PlatformIO`. Setup two project directories - one for transmitter and one for receiver.

```
pio project init --board uno
```

2) For source codes of transmitter and receiver, refer to the `tran` and `rec` respectively from

- https://github.com/gadepall/morse/codes/VLC
  for Visible Light Communication
- https://github.com/gadepall/morse/codes/IR
  for Baseband IR Communication
- https://github.com/gadepall/morse/codes/ModulatedIR
  for Modulated IR Communication

3) Copy the source code for the transmitter and receiver into `src/main.cpp` file in the respective project directories. The source codes are available in the above respective directories.
4) First upload the code to the receiver and then the transmitter. The text data to be transmitted can be input into the string in transmitter code.
5) Upload the code using the following command.

```
pio run -t upload
```